SYSTEM ANALYSIS DOCUMENTATION (SAD)

Stratatouille - AI Decision Spending Assistant

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This section introduces Stratatouille, a strategic AI-powered financial intervention designed to solve the chronic intuition-based spending habits of university students. While traditional trackers only look backward at past mistakes, Stratatouille uses predictive AI reasoning to act as a preventive guardrail for student finances.

1.1. Purpose:

Stratatouille is an AI-powered spending decision assistant designed for university students and young adults who struggle to manage their monthly budgets effectively. This System Analysis Documentation (SAD) outlines the technical scope, architectural decisions, and design rationale behind the development of Stratatouille for UMHackathon 2026.

The key elements covered in this document include:

1. ***Architecture**:*
    a. *A Flutter + Firebase mobile application backed by a FastAPI server and Z.AI GLM as the AI reasoning engine.*
2. ***Data Flows:***
    a. *How expense data flows from the user's mobile device, through the backend, gets stored in Firestore, and how the AI processes queries to return structured financial advice.*
3. ***Model Process:***
    a. *The end-to-end workflow of core features which are logging expenses, checking affordability, and receiving AI-powered spending insights.*
4. *Role of Reference:*
    a. *This document serves as a technical reference for developers, testers, and evaluators to understand the overall system design and component interactions.*

*1.2. **Background**:*

Students and young adults in Malaysia commonly face three critical barriers to smart financial management: limited analytical capabilities, reliance on intuition-based spending decisions, and fragmented financial data spread across multiple sources. These barriers lead to recurring problems such as overspending, unexpected budget shortfalls, and ongoing financial stress.

Stratatouille was conceived to solve this problem by providing an AI-powered assistant that is instant (records expenses in under 10 seconds), preventive (checks affordability before a purchase is made), and human-readable (delivers plain-language financial advice rather than raw spreadsheet data).

© UMHackathon 2026

Prior to Stratatouille, no unified solution existed that combined real-time budget tracking with conversational AI capabilities specifically tailored for the Malaysian student demographic. Existing budgeting apps lack context-aware intelligence and require heavy manual effort, which discourages consistent usage.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
University Students (Primary Users)	*Record daily expenses, query affordability before purchases, review spending summaries via dashboard*	*Sub-10-second expense logging, accurate affordability advice, plain-language AI responses, no financial jargon*
Development Team	*Build and maintain the Flutter app, FastAPI backend, Firebase integration, and Z.AI GLM prompting pipeline*	*Clear API contracts, modular codebase, well-defined prompt structures and response schemas*
QA Team	*Validate system behaviour across all user flows including expense logging, AI chat, and affordability checker*	*Defined request/response formats, GLM fallback behaviour specs, and edge case coverage*
Hackathon Evaluators	*Assess technical feasibility, AI integration quality, and real-world impact of the solution*	*Clear architecture diagrams, working demo, explainable AI integration, and measurable impact metrics*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Mobile Application (Android & iOS via Flutter)*
Architecture	*Client-Server with Cloud Backend and AI Service Layer*
Frontend	*Flutter (Dart) — cross-platform mobile app*
Backend	*FastAPI (Python) — RESTful API server*
Database	*Firebase Firestore — cloud NoSQL document database*
AI Engine	*Z.AI GLM — called via FastAPI as a managed service layer*

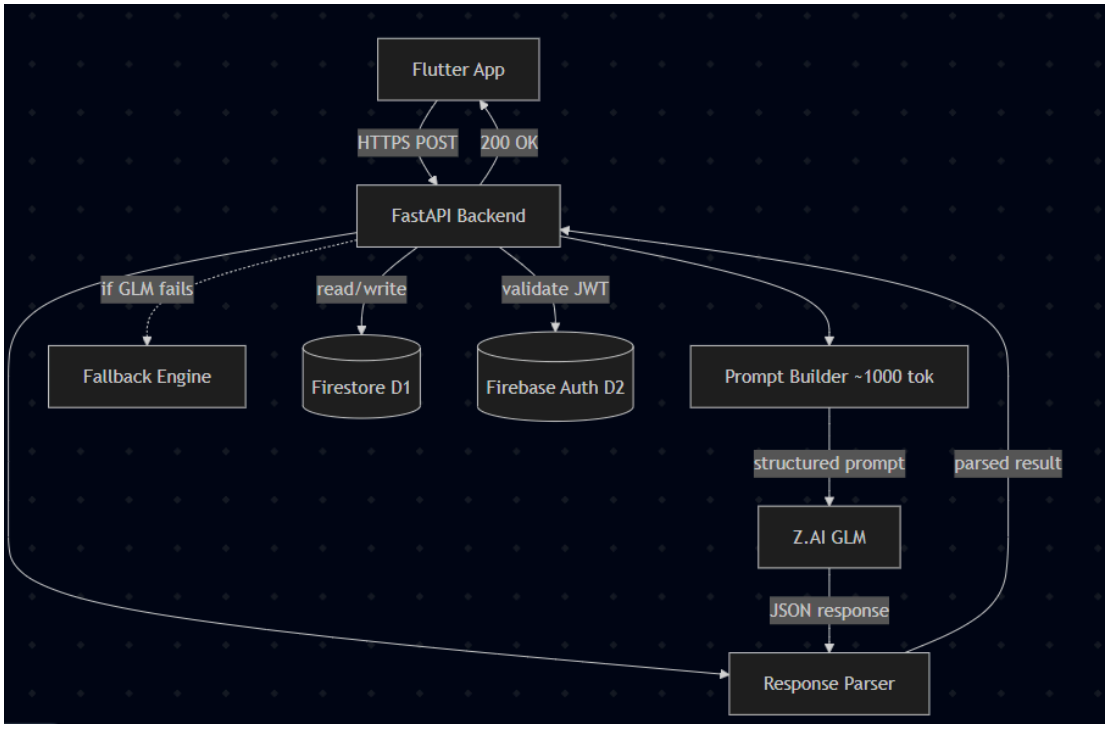
Stratatouille is structured as a client-server mobile application. The Flutter frontend communicates exclusively with the FastAPI backend over HTTPS REST calls. The backend is responsible for all business logic: it reads and writes expense data to Firebase Firestore, constructs structured prompts from user context, and calls the Z.AI GLM API to generate AI-powered financial advice. The GLM response is parsed and returned to the Flutter app as a structured JSON object containing three fields: Answer, Reason, and Based On.

2.1.2 LLM as Service Layer

Z.AI GLM is not a generic "AI module", it is integrated as a dedicated reasoning service layer within the backend. The FastAPI server acts as the orchestrator between the user's financial context and the GLM, ensuring that all prompts are well-structured, context-rich, and token-efficient.

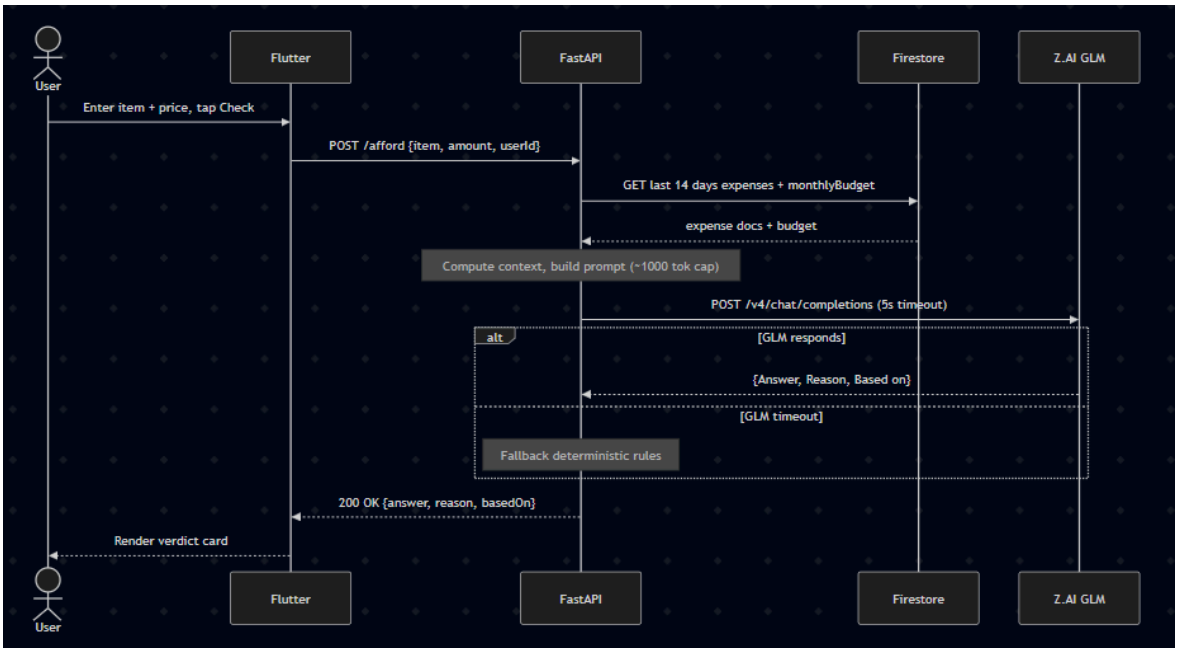
2.1.3 Dependency Diagram

© UMHackathon 2026



The diagram is organised into four horizontal layers. The Client layer contains the Flutter app, which communicates exclusively with the FastAPI backend via HTTPS POST requests — no direct database or AI calls are ever made from the frontend. The Backend layer contains three sub-components: the Prompt Builder, which assembles the ~1,000-token structured prompt from the user's 14-day expense context; the Response Parser, which extracts and validates the JSON output; and the Deterministic Fallback Engine, which triggers automatically if GLM is unavailable. The AI layer contains Z.AI GLM, called at POST /v4/chat/completions, which returns a structured three-field JSON (Answer, Reason, Based on). The Storage layer contains Firebase Firestore (userId-scoped read/write), Firebase Auth (JWT token validation on every FastAPI request), and a token budget tracker (~RM0.02 per interaction). No component in a lower layer initiates upward communication — all data flows are request-driven from the top.

2.2.2. Sequence Diagram



*This diagram walks through the **Affordability Check** feature end-to-end. Steps 1–2 cover the user triggering the check and the Flutter app issuing a `POST /afford` request to FastAPI with the item name, price, and `userId`. Steps 3–4 show FastAPI querying Firestore for 14-day expense history and the user's monthly budget, then computing derived values (total spent, remaining budget, daily average). Step 5 is the token enforcement gate — the prompt builder caps context at ~1,000 tokens by limiting the expense window. Steps 6–7 are the GLM roundtrip: FastAPI sends the structured prompt with a 5-second async timeout; GLM returns a JSON object with three fields. If GLM times out, the fallback engine takes over. Steps 8–9 show the response flowing back through FastAPI (validated) to Flutter, which renders the Answer as a status badge, the Reason as an explanation card, and the Based on as a data citation.*

2.2. Technological Stack

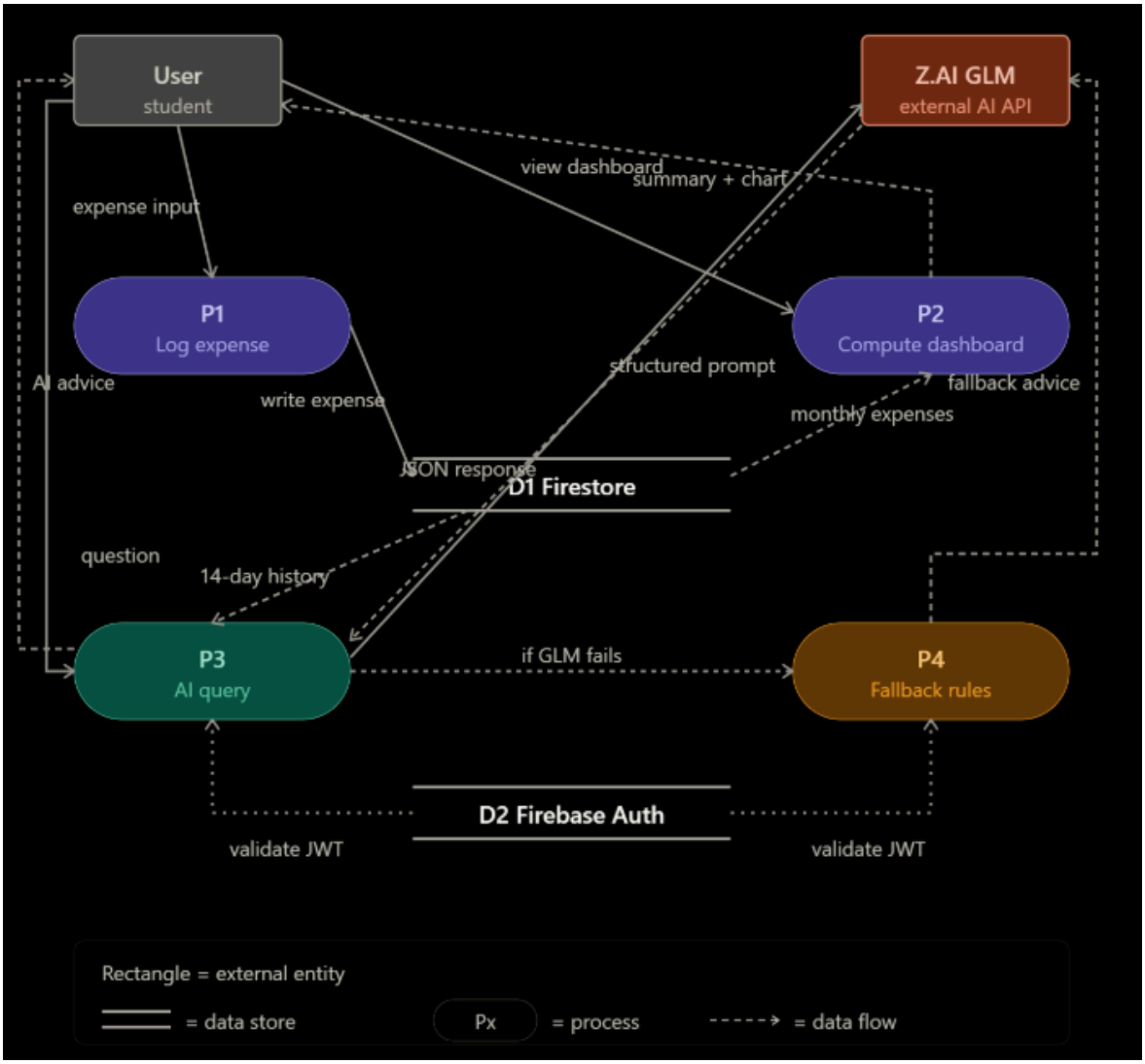
Layer	Technology	Justification
Frontend	*Flutter (Dart)*	*Cross-platform support for Android and iOS from a single codebase. Fast UI rendering and native-feeling performance critical for sub-10-second expense logging UX.*

Backend	*FastAPI (Python)*	*High-performance async Python framework. Ideal for orchestrating multiple async calls: Firestore reads, GLM API calls, and response parsing in a single request lifecycle.*
Database	*Firebase Firestore*	*Serverless NoSQL cloud database with real-time sync. Eliminates infrastructure management overhead during hackathon. Document-based structure maps naturally to expense records.*
AI Engine	*Z.AI GLM*	*Hackathon-provided LLM with strong reasoning capabilities. Called via FastAPI backend as a managed service. Returns structured JSON output via few-shot prompting.*
Authentication	*Firebase Auth*	*Pre-built authentication with Google sign-in support. Secure user isolation for expense data in Firestore.*
Hosting/Deployment	*Local FastAPI server (Hackathon scope)*	*FastAPI server runs locally and is accessible to the Flutter app during demo. Cloud deployment (e.g., Google Cloud Run) is planned as a post-hackathon enhancement.*

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)

© UMHackathon 2026



a. *The DFD follows Yourdon/DeMarco notation. External entities (rectangles) are the User and Z.AI GLM — they originate or consume data but are not controlled by the system. Processes (rounded shapes) are the four functions the system performs: P1 Log Expense, P2 Compute Dashboard, P3 AI Query & Response, and P4 Fallback Rules. Data stores (parallel lines) are D1 Firestore and D2 Firebase Auth.*

b. *Black hole / miracle check (DFD laws): No process is a black hole (receives input but produces no output) — P1 receives expense input and writes to D1; P2 reads D1 and outputs dashboard data to the user; P3 reads D1 and the GLM and returns AI advice; P4 receives a failure signal from P3 and outputs rule-based advice. No process is a miracle (produces output with no input) — every output arrow from every process has a corresponding input arrow. D2 (Firebase Auth) is a read-only store that validates the user's JWT at request time; no process writes new data to it during normal operation.*

© UMHackathon 2026

2.3.2. **Normalized Database Schema**

Collection: users/{userId}/expenses/{expenseId}

Field	***Type***	***Description***	***Constraints***
expenseId	*String (auto)*	*Unique document ID generated by Firestore*	*Auto-generated, immutable*
userId	*String*	*Firebase Auth UID of the owning user*	*Required, indexed*
amount	*Float*	*Expense amount in Malaysian Ringgit (RM)*	*Required, > 0*
category	*String (Enum)*	*Spending category*	*One of: Food, Transport, Bills, Shopping, Others*
description	*String*	*User-provided description of the expense*	*Required, max 200 chars*
timestamp	*Timestamp*	*Date and time the expense was recorded*	*Required, auto-set on creation*
createdAt	*Timestamp*	*Server-side creation timestamp*	*Auto-set by Firestore server time*

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	***Features***	***Description***

© UMHackathon 2026

1	*Expense Dashboard*	*Displays current month spending summary including: total spent, remaining budget, projected month-end balance, spending by category (with percentage bars), daily average spend, smart alerts (budget pace warnings), and a Budget vs. Actual comparison chart.*
2	*Add Expense*	*Allows users to log a new expense in under 10 seconds. Supports amount input (RM), free-text description, and category selection. Includes Quick Add chips for frequent transactions (e.g., RM10 Food, RM5 Bus). Shows a recent expense history list below the form.*
3	*AI Chat (Spending Assistant)*	*A conversational chat interface powered by Z.AI GLM where users can ask natural language questions about their spending (e.g., "Why am I overspending?", "What should I reduce?"). The AI responds with context-aware advice based on the user's real expense history.*
4	*Affordability Checker*	*Users enter an item name and price to receive an AI-powered verdict: can they afford it given their current budget pace and remaining funds? The response includes a clear Answer (Yes/No with nuance), a Reason, and a Based On citation of the data used.*

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	***Requirements***	***Implementation***
Performance	*Expense logging must complete in under 10 seconds end-to-end. AI Chat responses must render within 5 seconds under normal load.*	*Firestore writes are async and non-blocking. FastAPI uses async/await for GLM API calls. Flutter UI shows a loading indicator while awaiting AI response.*
Reliability	*Core expense logging and dashboard must function even when GLM is unavailable. Affordability Checker must degrade gracefully.*	*FastAPI implements a try/except fallback: if the GLM API call fails or times out (5-second timeout), deterministic rule-based logic is*

© UMHackathon 2026

		executed and returned in the same JSON format.
Scalability	*System should handle concurrent users without Firestore write contention or backend bottlenecks.*	*Firestore scales automatically as a serverless database. FastAPI can be horizontally scaled behind a load balancer for post-hackathon production deployment.*
Token Latency	*Z.AI GLM API calls must return within 3 seconds at p50. Timeout enforced at 5 seconds.*	*FastAPI enforces a 5-second async timeout on all GLM API calls. User-facing loading states are shown in the Flutter UI during inference. Prompt size is capped at ~1,000 tokens.*
Cost Efficiency	*Average token cost per user interaction must not exceed RM0.05.*	*Prompts are fixed-structure with bounded context (14-day expense window, ~1,000 tokens). Repeated affordability queries for the same item within 1 hour are cached in-memory by FastAPI to avoid redundant GLM calls.*
Maintainability	*The codebase must be modular with clear separation between Flutter UI, FastAPI business logic, and Firestore data access.*	*FastAPI routes are separated by feature (expenses, affordability, chat). Firestore access is abstracted into a dedicated data service layer. Flutter screens follow a provider-based state management pattern.*
Security	*User expense data must be isolated, no user can access another user's data.*	*Firebase Firestore security rules enforce userId-scoped read/write access. Firebase Auth tokens are validated on all FastAPI endpoints before any data access.*

3.3. Out of Scope / Future Enhancements

The following features represent the grand vision for Stratatouille but are not part of the UMHackathon 2026 submission:

a. *Bank account integration via Open Banking APIs (e.g., auto-import transactions from Maybank, CIMB)*

b. *Multi-currency support for students studying abroad or managing foreign exchange*

c. *Group expense splitting — shared budgets for housemates or friends*

d. *Recurring expense detection and automated monthly reminders*

e. *AI-powered savings goal recommendations based on spending patterns*

f. *Push notifications for budget alerts and overspending warnings*

g. *Export to PDF or Excel for financial reporting*

h. *Social spending comparison features (anonymised peer benchmarking)*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: For post-hackathon production deployment, the following rollout strategy is planned:*

4.1.1.1. *Phase 1 — Internal Testing (Week 1): The app is released to team members and close beta testers only. Core metrics monitored: GLM API error rate, average response latency, and Firestore write success rate.*

4.1.1.2. *Phase 2 — Grayscale Rollout (Week 2–3): The app is released to 10% of real users. Smart alert accuracy and affordability checker verdict correctness are monitored via user feedback signals.*

4.1.1.3. *Phase 3 — Full Release (Week 4+): If error rates remain below 1% and p95 latency remains under 5 seconds, the rollout is expanded to 100% of users.*

4.1.1.4. *A/B Testing: Two prompt variants for the Affordability Checker will be tested — one using a concise single-paragraph prompt vs. one using structured few-shot examples. The variant with higher user satisfaction ratings (tracked via thumbs up/down in the AI Chat UI) will be adopted as the default.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: The following emergency strategies are in place:*

© UMHackathon 2026

4.1.3. P1 Critical Trigger: If GLM API error rate exceeds 5% within a 15-minute window, the backend automatically switches to the deterministic fallback mode for all incoming requests. An alert is sent to the development team.

4.1.4. P2 Major Trigger: If Firestore write failures exceed 2% in any 5-minute window, the app displays a user-facing error message ('Unable to save expense. Please try again.') and retries the write with exponential backoff (3 retries, 1s / 2s / 4s delays).

4.1.5. Golden Release: Before any deployment, the last stable build is tagged as the Golden Release in the CI/CD pipeline. If a new deployment causes P1 triggers within 30 minutes, the automated pipeline rolls back to the Golden Release build.

4.1.6. Priority Matrix:

Priority	Trigger Condition	Action
P1 — Critical	*GLM API error rate > 5% in 15 minutes, or Firestore unavailable*	*Immediate automatic switch to deterministic fallback. Developer alert triggered. Rollback to Golden Release if issue persists > 30 minutes.*
P2 — Major	*Firestore write failure rate > 2% in 5 minutes*	*Retry with exponential backoff (3 retries). User-facing error message displayed. Developer notified within 10 minutes.*
P3 — Minor	*AI Chat response latency > 5 seconds at p95*	*Log slow queries. Review and optimise prompt size. Consider increasing GLM timeout threshold.*
P4 — Low	*Dashboard load time > 3 seconds*	*Investigate Firestore query performance. Add composite index on userId + timestamp fields.*

4.2. Monitoring:

4.2.1. API Health Checks: FastAPI server exposes a GET /health endpoint that returns server status and GLM connectivity status. Monitored every 60 seconds by an uptime monitor.

4.2.2. GLM API Latency Tracking: All GLM API calls are timed server-side. Response times are logged to a structured log file. Alerts are triggered if p95 latency exceeds 5 seconds over a 5-minute window.

4.2.3. Error Rate Monitoring: All HTTP 5xx responses from the FastAPI server are logged with request context (endpoint, payload size, GLM availability status). Error rate dashboards are reviewed daily.

© UMHackathon 2026

4.2.4. Token Usage Monitoring: Token count per GLM request is logged. Monthly token cost is tracked against the RM0.02/interaction target. If average cost exceeds RM0.05/interaction, prompt optimisation is triggered.

4.2.5. Firestore Usage Metrics: Firebase Console provides built-in monitoring of read/write operation counts and latency. Quota alerts are configured to notify the team before usage limits are reached.

4.3. Assumptions:

The following key assumptions were made during development and deployment planning:

a. *Users have a stable mobile internet connection (3G or above) on their devices when using the app, as real-time Firestore sync and GLM API calls require active connectivity.*
b. *The Z.AI GLM API is available and responsive within the 5-second timeout window for the majority of user interactions during the hackathon demo.*
c. *Users are university students or young adults aged 18–25 with a monthly budget in the range of RM500–RM3,000, which informs the default budget setting of RM2,000.*
d. *Expense categories (Food, Transport, Bills, Shopping, Others) are sufficient to cover the majority of student spending patterns without requiring custom categories.*
e. *The Firebase free tier (Spark Plan) is sufficient for the scale of usage during the hackathon demo period (estimated < 100 active users with < 500 Firestore reads/writes per day).*
f. *Users are willing to manually log expenses rather than requiring automatic bank transaction import, which is a post-MVP feature.*

4.4. External Dependencies:

Tool / Service	Purpose	Risks	Mitigation Strategy
Z Ai GLM API	Core AI reasoning engine — handles affordability checker reasoning, spending pattern analysis, and AI Chat question answering. Processes structured prompts and returns JSON-formatted financial advice.	HIGH — GLM unavailability directly impacts the AI features which are the core value proposition of the app.	Fallback deterministic rules engine implemented in FastAPI. If GLM returns an error or times out (5s), rule-based affordability check and generic spending advice are returned in the same JSON

© UMHackathon 2026

			format, ensuring the app remains functional.
Firebase Firestore	*Cloud NoSQL database for storing user expense records and budget settings. Provides real-time sync between app sessions.*	*MEDIUM — Firestore outages would prevent expense logging and dashboard loading.*	*Firestore has a 99.999% SLA. Retry logic with exponential backoff is implemented for write failures. Offline persistence is enabled in the Flutter Firebase SDK so previously loaded data remains viewable offline.*
Firebase Authentication	*User authentication and identity management. Issues JWT tokens used by FastAPI to authorise data access.*	*LOW — Firebase Auth is highly reliable and used only at login time.*	*Firebase Auth has a 99.95% SLA. Login failures are handled gracefully with a retry prompt in the Flutter app.*
Flutter SDK	*Cross-platform mobile framework for building the iOS and Android app from a single Dart codebase.*	*LOW — Flutter is a stable, mature framework.*	*Stable release channel used. No experimental features relied upon.*

5. Project Management & Team Contributions

5.1. Project Timeline:

Phase	Days	Activities
Phase 1 — Planning & Design	*Day 1–2*	*Problem statement analysis, feature scoping, UI/UX wireframing in Figma, technology stack finalisation, Firestore schema design, and GLM prompt strategy planning.*

© UMHackathon 2026

Phase 2 — Core Development	*Day 3–5*	*Flutter app scaffolding, Firebase Auth and Firestore integration, FastAPI server setup, Firestore CRUD endpoints for expense logging, and dashboard computation logic*
Phase 3 — AI Integration	*Day 6–7*	*Z.AI GLM API integration in FastAPI, prompt engineering and few-shot example design, structured JSON response parsing, AI Chat UI implementation, and Affordability Checker UI implementation.*
Phase 4 — Testing & Refinement	*Day 8–9*	*End-to-end testing of all user flows, GLM fallback logic validation, UI polish and UX improvements, bug fixes, and performance optimisation of Firestore queries.*
Phase 5 — Documentation & Submission	*Day 10*	*Documentation completion, pitch deck finalisation, demo video recording, code repository cleanup, and final submission.*

5.2. Team Members Role:

Member	**Role**	**Responsibilities**
Muhamad Firman Hakim bin Masri	*Project Lead & Backend Engineer*	*FastAPI server architecture, Z.AI GLM API integration, prompt engineering, structured JSON response parsing, GLM fallback logic implementation, and overall system integration.*
Nik Muhammad Syakir Bin Sukri	*Flutter Frontend Engineer*	*Flutter app architecture, all UI screens (Dashboard, Add Expense, AI Chat, Affordability Checker), Firebase Auth integration, Firestore real-time data binding, and state management.*
Aaron Hafidz Bin Mazrain	*Firebase & Data Engineer*	*Firestore schema design, security rules configuration, expense data queries and aggregation logic, dashboard computation algorithms, and budget tracking calculations.*
Muhammad Fariq bin Fauzi	*UI/UX Designer & Documentation Lead*	*Figma wireframes and UI design system, pitch deck creation, System Analysis Documentation (SAD), and demo preparation.*

5.3. Recommendations:

The following recommendations are proposed for scaling Stratatouille beyond the hackathon:

© UMHackathon 2026

- *Redis Cache for GLM Responses: Implement a Redis cache layer in FastAPI to cache GLM responses for identical affordability queries within a 1-hour window. This would reduce token costs by an estimated 30–40% for repeated queries.*
- *Cloud Deployment on Google Cloud Run: Deploy the FastAPI backend as a containerised service on Google Cloud Run for auto-scaling, managed HTTPS, and zero cold-start latency for production traffic.*
- *Firestore Composite Indexes: Add composite indexes on (userId, timestamp) and (userId, category, timestamp) fields to optimise dashboard aggregation queries as expense volume grows.*
- *Open Banking Integration: Integrate with Malaysian Open Banking APIs (BNM's Open API framework) to enable automatic transaction import from major banks (Maybank2u, CIMB Clicks), eliminating the need for manual expense logging.*
- *Push Notification Service: Implement Firebase Cloud Messaging (FCM) to deliver proactive budget alerts when spending pace is projected to exceed the monthly budget before month end.*
- *GLM Model Upgrade Path: Design the prompt pipeline to be model-agnostic, allowing easy switching to more capable or cost-efficient LLMs (e.g., GPT-4o, Claude) as the product scales and requirements evolve.*

© UMHackathon 2026